

## Infraestructura de Identidad Segura para la FIS

La computación distribuida permite centralizar las identidades de forma segura de una organización. En este proyecto, a través de la implementación de Kerberos, PKI, LDAP y alta disponibilidad, se diseñó la infraestructura para gestionar identidades de la FIS. El proyecto se desarrolló en WSL Ubuntu, con uso de Dockers artesanales y herramientas para monitorear su estado que se detallan a continuación.

La arquitectura del proyecto de directorio activo de identidad segura para la FIS se construyó sobre 4 nodos Linux, contenedores Docker que usan systemd real, con una imagen base jrei/sysmd-ubuntu. Esta decisión fue tomada para utilizar start/stop/enable como si fuera una máquina virtual de Virtual Box, con el beneficio de que se economiza RAM, CPU y almacenamiento. El primero de los nodos es el master que tiene la CA raíz autofirmada, el servidor LDAP master y el KDC principal de Kerberos. El nodo2 replica LDA con synrpl de modo automático y persistente para siempre estar sincronizado con el master y tener los mismos directorios, y también replica KDC recibiendo la base de datos con krop y kpropd. El tercero, nodo3, usa HAProxy para balancear la carga con el modo TCP y la aplicación web se desarrolló con el framework Flask, el cual usa TLS para proteger los certificados. Por último el nodo4 es el nodo que actúa como cliente Linux, un host, y también tiene las herramientas para monitoras los recursos del sistema, RAM, CPU y almacenamiento con Prometheus, Grafana y node\_exporter en los 4 nodos.

La conexión de estos nodos se realiza con una red puente (172.20.0.0/24) y resuelve automáticamente sus hostnames FIS como por ejemplo: ldap1.fis.epn.ec, kdc1.fis.epn.ec, con alias de Docker Compose. La gestión de identidades es centralizada por la aplicación web, en el momento que se crea un usuario desde el panel de administración, el backend realizado con Python da de alta la cuenta en LDAP, y en la misma transacción, el servicio de kerberos también en el backend crea el principal correspondiente con kadmin remoto autenticado por una keytab de servicio. Esto logra que ambos directorios estén sincronizados. En el caso de que la creación principal de Kerberos falle, LDAP revierte los cambios para mantener la sincronización.

La PKI se implementó con una CA raíz autofirmada generada con OpenSSL con el algoritmo ECDSA. Esta CA firma certificados de servidor ldap1, ldap2 y webserver. El directorio activo LDAP se configuró en el puerto 639 para que funcione sobre TLS. LDAP se instaló con OpenLDAP slapd y con el DIT dc=fis,dc=epn,dc=ec dividido en ou=People (cuentas inetOrgPerson/posixAccount/shadowAccount, con homeDirectory, loginShell y mail) y ou=Groups (posixGroup para estudiantes y profesores). Estos atributos posix hacen que las cuentas puedan ser usadas como cuentas Linux. El KDC se configuró para el real FIS.EPN.EC con principals de

usuarios de prueba (dpilataxi, jperez, malvan) y principals de servicio (ldap/ldap1.fis.epn.ec, http/webserver.fis.epn.ec).

La alta disponibilidad se resuelve por medio de tres capas independientes, LDAP, Kerberos y el balanceo y failover. LDAP replica constantemente el master-replica con syncprov en el nodo1 y en el consumidor con syncrepl de forma persistente en el nodo2. Para Kerberos el KDC secundario en nodo2, recibe su base de datos por propagación krop desde el nodo1 cada 60 segundos con un timer de systemd. Los clientes en el nodo3 y nodo4 listan ambos KDCS en /etc/krb5.conf por lo que si el principal cae el secundario asume el tráfico. Y por último el balance y failover opera con HAProxy en el nodo 3 en modo TCP, enrutado al puerto 636 LDAPS a ldap1 como server principal y ldap2 como backup monitoreado con healthcheck . El puerto 443 también se enruta a Flask, que termina el TLS con el certificado emitido por la CA.

El tráfico de LDAP está cifrado con TLS 1.2+ usando certificados ECDSA firmados por una CA propia, lo que elimina la transmisión de credenciales en texto plano. La cuenta de replicación, cn= replicator, tiene una ACL de solo lectura sobre el árbol. La autenticación de usuario es delegada a KERBEROS lo que evita el almacenamiento y transmisión repetida de contraseñas. Flask usa un formulario HTML que valida con kinit interno contra Kerberos, en lugar de negociar desde el navegador.

Los experimentos de inyección de fallos definidos en los requerimientos del proyecto se ejecutaron de forma exitosa y en las pruebas manuales se confirma que las lecturas de LDAP contra la réplica siguen disponibles aun cuando se diente slapd en el master, el cliente Kerberos redirige automáticamente al KDC secundario cuando el primario deja de responder sin tener que reconfigurar nada manualmente gracias a la lista de KDCs en krb5.conf. HAProxy redirige el trafico de LDAPS a ldap2, cuando el healthcheck TCP falla.

Enlace al repositorio en Github:

<https://github.com/DavidPilataxi/minildM/tree/main>